Learning GenAI Systems: The 20% That Matters

(The Capstone — How Everything Fits Together)

1 The One Reframe That Matters

A production generative-AI system is **not a model**. It is a *compound system* in which an LLM is one component among many. The single biggest mindset shift: stop thinking "which model do I call" and start thinking "what system do I build around the model." In production, even an agent is best understood as a distributed system in which the LLM happens to be the planner/executor — everything else (state, tools, retries, evaluation, safety) is ordinary, demanding engineering.

This document is the capstone for the whole stack you have learned. Each earlier tool is one box in the diagram below.

2 The Anatomy of a GenAI System

A typical production system contains these components, most of which you now have a tool for:

- **Serving / inference** — run the model with high throughput → *vLLM*.
- **Retrieval** — pull relevant context from your data → *RAG / LlamaIndex*.
- **Orchestration** — multi-step control flow, tools, memory → *LangGraph*.
- **The API layer** — expose it to clients → *FastAPI*, called via the *API* patterns.
- **Model customization** — task-specific behavior → *fine-tuning (LoRA/QLoRA)*.
- **Packaging & environment** — reproducible deploys → *Docker*, *uv*.
- **Routing** — send each query to the right handler/model by intent or difficulty.
- **Guardrails** — filter inputs and outputs for safety and structure.
- **Caching** — cut cost and latency on repeated work.
- **Evaluation & observability** — know whether it works and why.
- **Feedback loop** — use real usage to improve over time.

The last five have no single "tool" from the earlier documents — they are the cross-cutting discipline that turns a demo into a system, so this document focuses there.

3 The Build Decision (Escalate, Don't Jump)

When the base model isn't good enough, climb this ladder in order — each step costs more effort, so only ascend when the cheaper one plateaus:

1. **Prompt engineering** — clearer instructions, examples, output format. Free; try first.
2. **Retrieval (RAG)** — the problem is missing *knowledge*. Ground answers in your data.
3. **Fine-tuning** — the problem is missing *behavior/format/style*, not facts.
4. **Agentic / multi-step** — the task needs tools, planning, or several stages.

Most "the model is wrong" problems are solved at step 1 or 2. Reaching for fine-tuning first is the classic beginner mistake.

4 Routing and Model Choice

You do not use one model for everything. A router inspects each request and sends it to the cheapest capable handler: a small fast model for easy queries, a large model for hard ones, a specialized fine-tune for a narrow task, or a non-LLM path (a database lookup) when no generation is needed. This "small model by default, big model when needed" pattern is one of the largest cost levers in any system.

5 Hallucination: Prevention and Mitigation

Hallucination is the defining failure mode. Handle it in two layers:

- **Prevention:** ground responses in retrieved sources (RAG), prompt the model to acknowledge uncertainty rather than guess, and add output-verification steps that check claims before they reach the user.
- **Mitigation (for when it slips through):** cite sources so users can verify, show confidence indicators, and route high-stakes decisions through a human (human-in-the-loop).

Note the synergy: RAG is not just a knowledge tool, it is a primary hallucination-prevention mechanism.

6 Guardrails (Input and Output)

Guardrails are checks wrapped around the model:

- **Input guardrails** — block prompt injection, off-topic, or malicious requests before they reach the model.
- **Output guardrails** — filter unsafe content, redact confidential data, and enforce **structure** (e.g. validate the output against a JSON schema so downstream code can rely on it).

The hard truth about guardrails: most perform well on clean test sets but collapse under adversarial, out-of-distribution attacks — and adversarial conditions are exactly what production faces. Evaluate any guardrail against adversarial inputs, not just curated examples; clean-data benchmarks are misleading.

7 Evaluation (The Discipline That Separates Toys From Systems)

You cannot improve, or even safely ship, what you do not measure. Evaluation for GenAI goes beyond simple accuracy:

- **Build an eval set** — a fixed collection of representative inputs with expected qualities. This is your regression test for prompts, models, and retrieval.
- **Evaluate the two halves of RAG separately** — did retrieval fetch the right context, and did generation use it faithfully? (As in the RAG document.)
- **For agents, evaluate the whole trajectory, not just the final message** — was the tool choice correct, were the arguments valid, how many steps, what time/cost, did it comply with policy?
- **LLM-as-judge** — use a model to score outputs, but only with an explicit rubric and human auditing of the judge; an unaudited judge is just another source of error.
- **Ship evals in CI** — run them automatically on every change, with deterministic tool mocks, so a prompt tweak can't silently regress quality.

8 Observability / LLMOps

Traditional observability is logs, metrics, and traces. GenAI adds AI-specific dimensions. The goal of LLM observability is to **correlate user intent, prompts, model parameters, tool actions, and infrastructure signals so you can explain why the system produced a given output**. Track at minimum:

- **Cost** — token usage per request (input + output tokens are the bill).
- **Latency** — especially time-to-first-token (TTFT) for interactive use.
- **Quality** — ongoing scores from your eval set on real traffic.
- **Guardrail hits** — how often filters fire, and whether new failure categories appear.
- **Full traces** — the complete chain of prompts/tool calls for any request, so you can debug the "why."

Tools like LangSmith, Langfuse, MLflow, and others provide this tracing; the principle matters more than the vendor.

9 Cost and Latency Optimization

The main levers, roughly in order of impact:

- **Routing** — small model by default (see above).
- **Caching** — exact-match caching for identical requests; *semantic* caching to reuse answers to similar questions; prefix caching at the serving layer (vLLM).
- **Quantization** — serve in FP8/AWQ to cut VRAM and raise throughput (vLLM).
- **Batching** — continuous batching at the serving layer maximizes GPU utilization (vLLM).
- **Shorter context** — retrieve and rerank tightly; do not stuff the whole corpus into the prompt.

10 Security and Governance (Awareness)

For real deployments, build controls *into* the execution flow, not bolted on after: identity-based access to data and tools, secure API communication, data-residency rules, and auditable logs of model usage. Prompt injection makes this harder than classic app security — treat any text the model reads (including retrieved documents and tool outputs) as potentially adversarial.

11 The Reference Architecture (Putting It All Together)

A request flows through the system like this:

```
Client
  | (HTTP)                                <- API patterns
FastAPI endpoint                          <- FastAPI
  -> Input guardrail (block bad input)
  -> Router (pick model / path)
  -> Orchestrator                         <- LangGraph
       -> Retriever (context)             <- RAG / LlamaIndex
       -> LLM call                        <- vLLM serving your model/LoRA
       -> Tools (as needed)
  -> Output guardrail (safety + JSON schema)
  -> Cache write
  -> Response to client
        |
   Observability traces + eval scores     <- logged throughout
        |
   Feedback -> improve prompts/data/model (loop)
```

Everything packaged with *Docker* and dependencies managed with *uv*.

12 The Maturity Ladder

Systems evolve through stages: (1) prototype, (2) deployed with basic monitoring, (3) scaled with version-controlled prompts/configs and A/B testing, (4) enterprise scale with automated fine-tuning pipelines, guardrails, and cost optimization, (5) AI-native operations with continuous evaluation driving updates. Know where you are; don't build stage-4 infrastructure for a stage-1 prototype.

13 The Mental Model in One Line

A GenAI system is an LLM wrapped in retrieval, routing, orchestration, guardrails, caching, and an API — and **held accountable by evaluation and observability**. The model is the engine; the system is the car.

14 The Checklist for Any GenAI System

1. Define the task and an **eval set** before building.
2. Start with prompting; escalate to RAG, then fine-tuning, only as needed.
3. Add input/output **guardrails**; validate structured output against a schema.
4. Choose serving (vLLM) and an API layer (FastAPI); package with Docker/uv.

5. Add **routing** and **caching** for cost/latency.

6. Instrument **observability** (cost, latency, quality, traces) from day one.

7. Run evals in **CI**; close the **feedback loop** from real usage.

15 Capstone Practice Task (Uses the Whole Stack)

Build a minimal but complete document-QA system and operate it like a real one:

1. **Data:** build a RAG index over your documents (LlamaIndex), with retrieve-then-rerank.

2. **Serve:** run a model with vLLM behind its OpenAI-compatible API.

3. **Orchestrate:** use LangGraph so the agent decides when to retrieve vs answer directly, with one extra tool.

4. **Expose:** wrap it in a FastAPI `POST /ask` endpoint.

5. **Guard:** add an input check (reject empty/abusive queries) and an output check (the answer must cite at least one source; enforce a JSON response shape).

6. **Evaluate:** write 10 question/expected-answer pairs; score answers (exact match or LLM-as-judge with a rubric). Make this a script you can re-run.

7. **Observe:** log per-request token count, latency, and whether a guardrail fired.

8. **Package:** containerize with Docker, manage deps with uv.

9. **Iterate:** change the chunk size or top-k, re-run your eval, and confirm with numbers whether it improved.

Finishing this exercises every document in the set at once — and the final step (changing one thing and proving the effect with your own eval) is the single habit that most separates people who build GenAI systems from people who only demo them.